

EPFL CIVIL-127, Lab 5 solution

5.1

Note:

If you wrote the following code, you might end up having less than the `count_marmottes` elements in your set if you happen to generate the same point twice. Look at the solution below to see the correct way to initialize `self.marmottes`. It is subtle bugs like these that cause programs to be incorrect and software to misbehave.

```
self.marmottes = set()
for i in range(count_marmottes):
    self.marmottes.add((rand.randint(0, width-1), rand.randint(0, height-1)))
```

main.py

```
import random

from lab5.marmotte_sweeper.ui import MarmotteSweeperUI

# Code to create an instance of MarmotteSweeperUI and start
# the game.
# Adjust difficulty by changing the grid size and number of marmottes.
# Adjust the cell_size depending on the screen resolution.

random.seed(2) # for testing purpose
game = MarmotteSweeperUI(9, 9, 15, 64)
game.start()
```

logic.py

```
from abc import ABC, abstractmethod
import enum
import random

class Cell(enum.Enum):
    UNREVEALED = 1
    REVEALED = 2
    MARMOTTE = 3
    FLAGGED = 4

type coord = tuple[int, int]
```

```

class Rand(ABC):
    @abstractmethod
    def randint(self, a: int, b: int) -> int:
        pass

class RealRand(Rand):
    def randint(self, a: int, b: int) -> int:
        return random.randint(a, b)

class MarmotteSweeper:
    """
    Class that handles the game state. The code tracks:
    - positions of all the marmottes
    - positions which have been revealed
    - positions which have been flagged
    - mark a cell as revealed
    - mark/unmark a cell as flag

    All coordinates are grid-based since this class only deals
    at the internal logic level and doesn't know anything about the UI.
    """

    def __init__(self, width: int, height: int, count_marmottes: int, rand: Rand):
        self.width = width
        self.height = height
        self.revealed: set[coord] = set()
        self.flagged: set[coord] = set()
        self.marmottes: set[coord] = set()
        while len(self.marmottes) < count_marmottes:
            x = rand.randint(0, width - 1)
            y = rand.randint(0, height - 1)
            self.marmottes.add((x, y))

    def cell(self, pos: coord) -> Cell:
        """
        Returns information about the cell at x, y
        """
        if pos in self.revealed:
            if pos in self.marmottes:
                return Cell.MARMOTTE
            else:
                return Cell.REVEALED
        if pos in self.flagged:
            return Cell.FLAGGED

```

```

        return Cell.UNREVEALED

def flag(self, pos: coord) -> None:
    """
    Toggle flag at x, y.
    x, y are grid coordinates.
    """
    if pos in self.flagged:
        self.flagged.remove(pos)
    else:
        self.flagged.add(pos)

def reveal(self, pos: coord) -> None:
    """
    Reveal position (x, y).
    x, y are grid coordinates.
    """
    self.revealed.add(pos)

```

ui.py

```

import os
import pygame
from lab5.marmotte_sweeper.logic import Cell, MarmotteSweeper, RealRand

class MarmotteSweeperUI:
    """
    Code responsible for instantiating MarmotteSweeper and handling
    the user interface.
    """

    def __init__(self, width: int, height: int, count_marmottes: int, cell_size: int):
        """
        width and height are the size of the game (in cells)
        count_marmottes is how many marmottes the game should have
        cell_size is the size of each cell in screen pixels
        """
        self.width = width
        self.height = height
        self.cell_size = cell_size
        self.marmotte_sweeper = MarmotteSweeper(
            width, height, count_marmottes, RealRand()
        )

        # Load images from assets/ directory

```

```

# type1 | type2 is used to describe the union of two types. You can also
# use the Any type to express that a type is mixed.
self.images: dict[Cell | int, pygame.Surface] = {}
for i in range(9):
    self.images[i] = pygame.image.load(os.path.join("assets", str(i) + ".png"))
self.images[Cell.MARMOTTE] = pygame.image.load(
    os.path.join("assets", "marmotte.png")
)
self.images[Cell.FLAGGED] = pygame.image.load(
    os.path.join("assets", "flag.png")
)
self.images[Cell.UNREVEALED] = pygame.image.load(
    os.path.join("assets", "blank.png")
)

# Resize images if needed
for k, v in self.images.items():
    self.images[k] = pygame.transform.smoothscale(
        v, (self.cell_size, self.cell_size)
    )

# Setup pygame and create a window
pygame.init()
self.window = pygame.Window(
    "Marmotte Sweeper", (width * cell_size, height * cell_size)
)

def draw(self) -> None:
    """
    Draws every cell of the grid.
    """
    surface = self.window.get_surface()
    for x in range(self.width):
        for y in range(self.height):
            cell = self.marmotte_sweeper.cell((x, y))
            image = self.images[cell]
            # convert grid positions to pixels
            surface.blit(image, (x * self.cell_size, y * self.cell_size))
    self.window.flip()

def start(self) -> None:
    """
    Starts the pygame event loop. Returns when the window is closed.
    """

```

```

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            return

        if event.type == pygame.MOUSEBUTTONDOWN:
            x, y = event.pos
            # convert pixels to grid positions
            x = x // self.cell_size
            y = y // self.cell_size
            self.marmotte_sweeper.flag((x, y))

    self.draw()
    pygame.Clock().tick(60)

```

5.2

test_logic.py

```

import unittest

from lab5.marmotte_sweeper.logic import Cell, Rand, MarmotteSweeper

class FakeRand(Rand):
    def __init__(self):
        self.i = 0

    def randint(self, a: int, b: int) -> int:
        self.i += 1
        return (self.i % (b - a + 1)) + a

class TestMarmotteSweeper(unittest.TestCase):
    def setUp(self) -> None:
        # 3x3 grid with Marmottes at 0,1 and 1,2
        # . . .
        # @ . .
        # . @ .
        self.marmotte_sweeper = MarmotteSweeper(3, 3, 2, FakeRand())

    def test_get_cell(self) -> None:

```

```
self.assertEqual(Cell.UNREVEALED, self.marmotte_sweeper.cell((0, 0)))

def test_flag(self) -> None:
    self.marmotte_sweeper.flag((0, 0))
    self.assertEqual(Cell.FLAGGED, self.marmotte_sweeper.cell((0, 0)))
    self.marmotte_sweeper.flag((0, 0))
    self.assertEqual(Cell.UNREVEALED, self.marmotte_sweeper.cell((0, 0)))

def test_reveal(self) -> None:
    self.marmotte_sweeper.reveal((2, 0))
    self.assertEqual(Cell.REVEALED, self.marmotte_sweeper.cell((2, 0)))
    self.assertEqual(Cell.UNREVEALED, self.marmotte_sweeper.cell((1, 1)))

if __name__ == "__main__":
    unittest.main()
```